

Achieving Privacy-Preserving Outsourced SVM Training with Non-linear Kernel

Yuandong Xie*, Jingcheng Zhao*, Bin Zhu*, Jian Li*, Ruidong Li† Kaiping Xue*‡

* School of Cyber Science and Technology, University of Science and Technology of China, Hefei, Anhui 230027, China

† College of Science and Engineering, Kanazawa University, Kakuma, Kanazawa 920-1192, Japan

‡Corresponding author, kpxue@ustc.edu.cn

Abstract—Cloud-based Support Vector Machine (SVM) is a powerful technique for decision-assistance service. However, training data and models of SVM contain sensitive information, outsourcing these data to clouds may lead to severe privacy leakage. To address the privacy issue of SVM, many works focus on outsourced privacy-preserving SVM training. However, these works cannot support training SVM with non-linear kernel. This limitation renders these methods impractical for real-world scenarios where datasets are usually non-linear. In this paper, we propose a privacy-preserving SVM training scheme with support to non-linear kernel. Specifically, we redesign a gradient descent for SVM with kernel, which supports efficient kernel SVM training. We design basic computation protocols using secret sharing to achieve privacy preservation in outsourced SVM training. Additionally, we construct an incremental learning approach to support continuous data inflow. This approach is capable of reducing computational overhead significantly in practical scenario. Security analysis and efficiency evaluation illustrate that our proposed scheme achieves superior accuracy and less computation overhead compared to existing works, while also preserving privacy of training data and trained SVM model.

Index Terms—Privacy-Preserving, Outsourced SVM, Kernel Function, Incremental Learning

I. INTRODUCTION

Outsourcing Support Vector Machine (SVM) in clouds is gaining significant attraction in various fields [1], [2]. With the advancement of cloud computing, numerous institutions opt to train SVM models using cloud platforms. As a crucial factor for training an accurate SVM model, kernel function is widely adopted [3]. Incorporating non-linear kernels in outsourced SVM training could yield substantial performance improvements over linear SVMs. In practical outsourced SVM training, training data may be generated steadily, incremental learning [4] could notably decrease the computation overhead cost by continuously arriving data.

However, outsourcing SVM training data and models may raise concerns about privacy. The training data as well as the trained SVM models contain sensitive information such as patients' medical records or employees' salary information of data owners. Outsourcing these data to untrustworthy clouds could result in severe privacy leakages. Privacy leakage concerns have significantly hindered the practical application of outsourcing SVM across various scenarios. Such a privacy issue has led researchers to investigate privacy-preserving outsource SVM training methods [5], [6].

Unfortunately, existing methods cannot address the issue of training non-linear kernel SVM while ensuring privacy. Training schemes implemented by existing approaches [5], [6] do not support introducing non-linear kernel since the algorithms they use are not compatible with non-linear kernel. There are also several works achieving non-linear kernel SVM on inference stage [7], [8], yet these works could not be directly applied to training process since SVM training and inference implement totally different algorithms. To the best of our knowledge, no previous work has solved the privacy issue of training non-linear kernel SVM.

Moreover, existing approaches also encounter difficulties in supporting incremental learning under privacy preservation [5], [6]. When new data arrives, the existing scheme necessitates retraining with the entire dataset, leading to substantial computational overhead. Given that such scenarios are prevalent in the real world (e.g., continuously operational institutions like hospitals or banks), there is a need to minimize overhead in such situations. However, existing works also struggle to achieve incremental learning without modification.

To address the above issues, we propose a privacy-preserving SVM training scheme that supports non-linear kernel. Specifically, We design a gradient descent for SVM that could introduce nonlinear kernel function and combine it with secret sharing. Additionally, we propose an incremental learning paradigm based on our training scheme. When data arrive steadily, the incremental learning paradigm is capable of training with only newly arrives data. The main contributions of our work can be summarized as follows:

- 1) We design basic secure protocols and construct a privacy-preserving SVM training scheme, which is capable of training SVM with non-linear kernel function.
- 2) Based on our proposed training scheme, we propose an incremental learning scheme for SVM, which enables SVM to train with incremental data and prevents SVM from retraining with the whole dataset.
- 3) We evaluate the performance and security of our scheme and compare them with prior works. Our scheme achieves better accuracy and less computation overhead and protects privacy of training data and trained model.

The remainder of this paper is organized as follows. In section II, some related literature is given. We introduce system model, security model and design goals in section

III. In section IV, we briefly review some preliminaries. Our proposed scheme are introduced in section V, then, in section VI and section VII, we respectively analyze the security and performance of our paradigm. Finally, a conclusion of this paper is given in section VIII.

II. RELATED WORKS

SVM have gained significant popularity as a powerful and relatively accurate machine learning model [3]. In this section, we review the existing literature on privacy-preserving SVM training approaches.

Laur *et al.* [9] proposed a privacy-preserving scheme where multiple parties own horizontally partitioned data and communicate with each other during every training phase to compute an SVM model globally. Yu *et al.* [10] proposed a privacy-preserving SVM training scheme in which all participants compute a global Gram matrix using their horizontally partitioned data with multi-party computation. The SVM model is then trained locally by each party in plaintext. However, publicizing the Gram matrix causes privacy leakage since the Gram matrix contains input of participants. To address this issue, [11] adopted the idea of a random kernel, which replaces part of parameter with a random number when calculating the kernel. The random kernel approach helps to reduce the privacy leakage but lead to descent in inference accuracy.

In summary, these approaches can preserve the privacy of training data, but fail to protect the privacy of trained model. Training SVM locally also requires relatively excessive computational resources in each participants, which is usually limited in practice. One possible solution to this problem is to securely outsource the training phase.

Liu *et al.* [6] proposed a privacy-preserving outsourced SVM scheme. They introduced a special homomorphic encryption, the Distributed Two Trapdoors Public-Key Cryptosystem (DT-PKC), achieving secure SVM training outsourcing with less user burden. However, their SVM training mechanism is Sequential Minimal Optimization (SMO), which involves a large number of logical computations that are unfriendly to privacy-computing. Wang *et al.* [5] improved the training process by applying a Gradient-Descent algorithm instead of SMO. Using GD in training SVM is privacy-computation friendly and incurs less computational overhead, [5] also solve the problem of plaintext overflow problem.

To summarize, these works only support the training of linear kernels, while most SVMs are equipped with a Gaussian kernel. The inference accuracy of these SVMs could be very low when faced with datasets that are not compatible with linear kernels, making them impractical in real-world scenarios. Achieving both privacy-computationally friendly operations and non-linear kernel is challenging, which motivates us to propose a new privacy-preserving SVM training scheme with non-linear kernel in this paper.

III. PROBLEM STATEMENT

In this section, we propose the system architecture, threat model and design goals of our scheme.

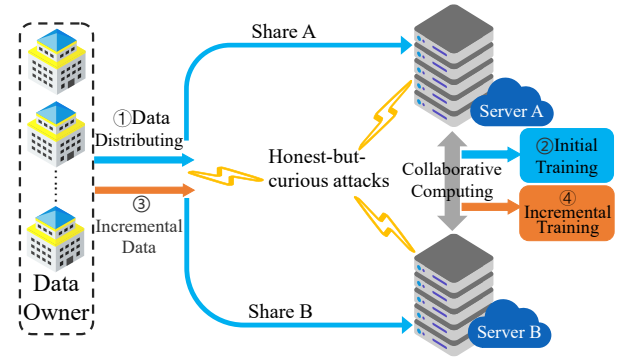


Fig. 1: Overview of system architecture

A. System Architecture

We consider the scenario where multiple data owners aim to train a kernel SVM without leaking any privacy. As shown in Fig. 1, there are two types of participants.

Cloud Servers: There are two non-colluding cloud servers CS_A and CS_B in our architecture. They are assumed to have sufficient computational and communication resources. The training process is carried out between the two cloud servers to obtain a kernel SVM model.

Data Owner: The data owner holds the training data, e.g., multiple hospitals hold various data of patients. They aim to train an SVM classifier without leaking any of their privacy. They are often institutes that keep running, which generates data continuously.

B. Threat Model

In our threat model, we assume the two cloud servers are Honest-but-Curious, which means they strictly follow the protocol while retaining interests in data owners' privacy. This is a common model used in previous works [12], [7]. Aside from the two servers, we assume all data owners are also Honest-but-Curious, which means they strictly follow the protocol but tries to access the trained model if possible. We assume the communication between all parties utilizes secure channels, which means that malicious adversaries cannot decode the secure channel to learn the content of the communication.

C. Design Goals

Privacy: The proposed scheme should ensure that cloud servers cannot learn any information about the training data. Besides, trained model should not be leaked to cloud servers or data owners.

Kernel: The proposed scheme should be able to introduce kernel function to SVM training and achieves better accuracy compared to prior works on real-world datasets.

Incrementality: The proposed scheme should support incremental learning and requires only a small amount of computation when training with newly added data.

Efficiency: The proposed scheme should be efficient in both computation and communication, and the computation cost of data owners should be as little as possible.

IV. PRELIMINARIES

We first sketch the definition of SVM and necessary preliminary concepts of secret sharing, and then introduce the ideology of incremental learning.

A. Support Vector Machine(SVM)

Training an SVM requires n training instances $(\vec{X}_1, y_1), \dots, (\vec{X}_n, y_n)$, where $\vec{X}_i$ represents the training data with m -dimensions and y_i is the corresponding label to training data, usually subject to y_i in $(1, -1)$. The SVM classifier attempts to determine a hyperplane between the positive samples and negative samples with a maximized soft margin. The training process of SVM is equivalent to solving the following dual problem, i.e.,

$$\text{Min}_{\alpha} \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j X_i^T X_j - \sum_{i=1}^n \alpha_i,$$

subject to $\sum_{i=1}^n \alpha_i y_i = 0$. Introducing kernel function $K(\vec{x}_i, \vec{x}_j)$ to replace $X_i^T X_j$ in this dual problem could significantly increase the inference accuracy since many realistic datasets are not linear-separable. Notice that $X_i^T X_j$ is also known as linear kernel. The Gaussian kernel is the most commonly used, which is denoted as:

$$K(\vec{x}_i, \vec{x}_j) = \exp\left(-\frac{\|\vec{x} - \vec{z}\|^2}{2\delta^2}\right).$$

The SVM's inference function is denoted as $h(\vec{t}) = \text{Sign}(\sum_{i=1}^n \alpha_i y_i K(\vec{x}_i, \vec{t}) + \beta)$, where $\vec{t}$ is the m -dimensions input and $\beta = \frac{1}{|S|} \sum_{i \in S} y_i - w^T x_i$, $S = x_i | x_i \in SV$.

B. Additive Secret Sharing

Additive Secret Sharing. Secret sharing is a classic cryptographic tool in which multiple parties hold different shares of an element x , denoted as $[x]^A, [x]^B$. In our scheme, arithmetic operation over secret sharing is defined over an integer ring Z_{2^k} and both input and output are kept in the form of shares, the shares on CS_A and CS_B can be denoted as $[x]^A, [x]^B$, where $[x]^A + [x]^B = x$.

Beaver's triple. To decrease the rounds of interaction and computation overhead in some operation when using secret sharing, Beaver *et al.* [13] proposed a technique that generates triples with Oblivious Transfer (OT). The generated triples are then delivered to the two cloud servers and are used to accelerate arithmetic operation, as proved in [14]. Such triples are data independent and hence can be generated during the offline phase [15].

C. Incremental Learning

Incremental learning is a machine learning strategy that updates models as new data becomes available, without requiring complete retraining. Leveraging previously acquired knowledge, it uses new data for model adjustments, reducing computational overhead. General incremental learning can be denoted as: $w_{t+1} = f(w_t, D_t)$, where w_t and D_t represents current parameters and incremental data at time step t and f is training function. This approach is efficient when data continuously arrives as it only trains new data.

V. PROPOSED SCHEME

A. Overview

In this section, we introduce our proposed scheme of secure kernel SVM training. The building blocks of our proposed scheme are designed based on secret sharing, which contains the following parts: Privacy-Preserving Gaussian Kernel Function, Privacy-Preserving Comparison and Privacy-Preserving Multiplication. Prior to training, All data owners locally split their data x_i and Corresponding class label y_i into two additive secret shares, which subjects to $x_i = [x_i]^A + [x_i]^B, y_i = [y_i]^A + [y_i]^B$. These data are then distributed to two cloud servers with the secure channel mentioned above. Upon receiving training data, two cloud servers run our proposed training scheme to train a SVM with non-linear kernel. After initial model training, system turns into incremental learning, once new data is provided, our scheme runs a lightweight incremental learning to train with only new data.

B. Model Training

Our proposed training scheme is Secure Kernel SVM Training, which is illustrated in Algorithm 1.

Algorithm 1: Secure Kernel SVM Training

Input : Training data $[x_i]$, corresponding label $[y_i]$,
Incremental data and label $[x_{a_j}], [y_{a_j}]$,
iteration limits $T_0, [THS_0]$ and $T_a, [THS_a]$

Output: Parameters $[U]$

Initial Training: $[U_0] = \text{PPGD}([x_i], [y_i], T_0, [THS_0])$

Incremental Learning:

if New data x_{a_j}, y_{a_j} arrives:

Calculates Kernel:

$K = \text{PPGKF}([x], [x_{a_j}])$

Judge if new data should update:

if $-[y_{a_j}][u]^T[K] + [b] > 1$ **then**

$[grad] \leftarrow [grad] - c[y_i][k_i]$

$[cost] \leftarrow [cost] + 1 - [y_i]([u]^T[k_i] + [b])$

end

$[u] \leftarrow [u] - \eta \cdot [grad]$

$t \leftarrow t + 1$

Repeat until $[cost] < [THS_a]$ or $t < T_a$

Secure Kernel SVM Training begins by training a SVM model with training data using Privacy-Preserving Gradient Descent (PPGD). Limits of iterations in initial training is relatively huge as the size of training data is large.

The initial training implement PPGD to train SVM, which inherits the idea of gradient descent. The main process of PPGD is described in Algorithm 2. Algorithm 2 calculates parameters of SVM in the form of secret sharing. It inherits the ideology of gradient descent and iteratively calculates the gradient of each training data, if the inference result of training data is not correct, parameters u will be decreased by $\eta * grad$, η here represents learning rate of this algorithm, which corresponds to the scale of the decreased gradient. $Cost$ and t are both used to restrict the times of iteration. Normally

Algorithm 2: Privacy-Preserving Gradient Descent (PPGD)

Input : Training data $[x_i]$, corresponding label $[y_i]$, iteration limits T and $[THS]$

Output: Parameters $[U]$

Initialize: CS_A and CS_B : randomly set $[u]$

$[K] = \text{PPGKF}([X], [x_i])$

while $t < T$ **or** $[cost] < [THS]$ **do**

$[grad] = [K] \cdot [u]$

for $i = 1$ **to** n **do**

if $-[y_i][u]^T[k_i] + [b] > 1$ **then**

$[grad] \leftarrow [grad] - c[y_i][k_i]$

$[cost] \leftarrow [cost] + 1 - [y_i]([u]^T[k_i] + [b])$

end

end

$[cost] \leftarrow [cost] + 0.5[u]^T[K][u]$

$[u] \leftarrow [u] - \eta \cdot [grad]$

$t \leftarrow t + 1$

end

the algorithm terminates when $cost \geq THS$, yet $cost$ could be small in specific scenarios. Under this situation, a limitation towards rounds of iteration is realized by judging if $t < T$.

Privacy-Preserving Gaussian Kernel Function is used to calculate the gaussian kernel of training data, as illustrated in Algorithm 3. We will mainly discuss gaussian kernel since it is the most commonly used non-linear kernel. PPGKF takes the input of two training data X_i, X_j and then reshapes them as $X_i^A - X_j^A - m_A, X_i^B - X_j^B - m_B$. CS_A and CS_B then calculates r_A, r_B , in fact, $r_A + r_B = 2(X_i^A - X_j^A)(X_i^B - X_j^B)$. Then, CS_B sends t^A to CS_A , after which CS_A and CS_B locally computes km^A, km^B . They are multiplicative secret shares of kernel function and $km^A * km^B = (e^{-\gamma})^{P_b + t^A + (X_i^A - X_j^A)^2 + r_A} = (e^{-\gamma})^{P_b + (X_i^B - X_j^B)^2 - P_b + r_B + (X_i^A - X_j^A)^2 + r_A} = e^{-\gamma(X_i - X_j)^2}$, which is the kernel function between X_i and X_j . However, since the kernel function is distributed in multiplicative form, PPGKF then transforms it in the form of additive share by calculating $k_A, k_B = \text{PPM}(n_A, km^B - n_B, km^A - n_A, n_B)$. k_A, k_B is computation result of Algorithm 3. Parameter γ here is a tuning argument and contains no information about training data, hence we assume the argument is known to both servers and is stored in the form of plaintext.

After initial training, incremental learning is activated whenever new data is added. Incremental learning first calculates the kernel function of new data, then iteratively calculate the gradient of current model and decide if current model need to be updated. If current model should be updated, the parameter u will be decreased with current gradient. It is worth noting that incremental learning uses a small limit of iterations T_a and THS_a and a small learning rate to train with only new data, which makes incremental learning efficient in computation.

Other basic computation protocols in our scheme such as privacy preserving multiplication and comparison are com-

Algorithm 3: Privacy-Preserving Gaussian Kernel Function (PPGKF)

Input : CS_A inputs $[X_i]^A, [X_j]^A$, CS_B inputs $[X_i]^B, [X_j]^B$

Output: CS_A outputs $[K_{i,j}]^A$, CS_B outputs $[K_{i,j}]^B$

CS_A : randomly select integer ma

Send to CS_B : $[X_i]^A - [X_j]^A - ma$

CS_B : randomly select integer mb

Send to CS_A : $[X_i]^B - [X_j]^B - mb$

CS_A and CS_B : $[r]^A, [r]^B = \text{PPM}(2ma, [X_i]^B - [X_j]^B - mb, mb, 2([X_i]^A - [X_j]^A - ma))$

CS_B : randomly select integer pb

Send to CS_A : $[t]^A = ([X_i]^B - [X_j]^B)^2 - pb + [r]^B$

CS_A : $[km]^A = (e^{-\gamma})^{[t]^A + ([X_i]^A - [X_j]^A)^2 + [r]^A}$

randomly select integer na

Send to CS_B : $[km]^A - na$

CS_B : $[km]^B = (e^{-\gamma})^{pb}$

randomly select integer nb

Send to CS_A : $[km]^B - nb$

CS_A and CS_B : $[k]^A, [k]^B = \text{PPM}(na, [km]^B - nb, [km]^A - na, nb)$

monly used in other works. The Privacy-Preserving Multiplication (PPM) with beavers' triple in our scheme follows the idea of [16], which calculates the multiplication between two data. As for Privacy-Preserving Comparison (PPC), we follow the idea of [17], which uses privacy-preserving addition to calculate the subtraction result $[v] = [x] - [y]$. After that, CS_A and CS_B use a bit extraction protocols to obtain $MSB(v)$, which is Most Significant Bit of v . Then, if $x > y$, the restored result of $MSB(v)$ is 1, otherwise it is 0.

VI. SECURITY ANALYSIS

A. Security of Training Data

Theorem 1: Assuming all cloud servers satisfy our threat model, the privacy of training data is protected.

Proof. Training data are split into two servers distributedly among all data owners with secure channel. Hence, during data outsourcing, training data is protected. During SVM training, the information exchange only occurs in Algorithm 3, PPM and PPC. Other algorithms such as Algorithm 1, 2 are designed based on the above three algorithms. Hence, we first analyze the security of these algorithms, then, we demonstrate training data is protected during training.

(1) Security of PPM and PPC: Since protocols of privacy-preserving multiplication and privacy-preserving comparison follow the ideology of existing works, the security proof is given in [16], [17].

(2) Security of Algorithm 3: Input of privacy-preserving gaussian kernel function is shares of two data samples. Since two servers are equivalent, we only analyze CS_A . During the whole process, CS_A receives $X_i^B - X_j^B - m_B, (X_i^B - X_j^B)^2 - P_b + r^B, km^B - n_B$, and holds m_A, r_A, km^A, n_A, k_A . To reconstruct any privacy including $X_i^B - X_j^B, (X_i^B - X_j^B)^2 +$

r^B, km^B, m_B, P_b or n_B is required. However, since the servers are semi-honest and follow the protocol as specified, none of m_B, P_b, n_B is known to CS_A . Therefore, assuming all cloud servers satisfy our threat model, we can conclude that the privacy of training data is protected. Hence, the privacy of computation result of kernel function is protected. It is noteworthy that all intermediate computation result that CS_A received is protected by a share of random number, e.g., ma, mb, pb, na, nb .

Algorithm 1, 2 are constructed from our basic computation protocols, and the input and output of these algorithms are secret shares that do not reveal any information. Hence, Algorithm 1, 2 are also privacy preservation. Since computation result after each protocol are stored in two cloud servers separately. Any server is incapable of reconstructing any information related to training data without collusion. Hence, under our threat model, the privacy of training data is protected.

B. Security of Trained Model

Theorem 2: Assuming all cloud servers satisfy our threat model, the privacy of trained model is protected.

Proof. In our scheme, the trained model is stored in the form of secret shares in two cloud servers, neither one of the servers nor the data owners are capable to learn anything about the trained model. Since we assume two cloud servers are Honest-but-Curious, no interaction except our scheme is allowed. Hence, the security of trained model is protected as long as PPGD is privacy-preserving and does not reveal any information, as proved above. Thus, the trained model is protected in face of data owners and cloud servers.

VII. PERFORMANCE EVALUATION

In this section, we evaluate the performance of our scheme in terms of accuracy and computation overhead, we also compare the efficiency of our proposed scheme with POD framework in [6] and EPoSVM in [5] since the idea of proposing secure outsourced SVM training in POD and EPoSVM is the same as ours. Aside from comparative evaluations, we also test computation overhead in realistic scenario where data arrives steadily. The configuration is a computer with intel core i5-10500 CPU 3.10 GHz and 12-GB RAM, and using python and java as the programming language. The dataset is the UCI cardiography dataset which consists of 21 features each sample and 1 corresponding label.

A. Accuracy Evaluation

In this part, we evaluate the accuracy of our proposed work, the training set and test set are both from the UCI dataset, in the experiment, we set $T < 200$ and $cost < 10$, which are tested under plaintext.

Accuracy under different numbers of samples: We measure the accuracy of trained model with different numbers of training samples, Fig.2(a) illustrates the accuracy of our proposed scheme along with POD and EPoSVM. Our scheme achieves the highest accuracy of approximately 0.912 when the

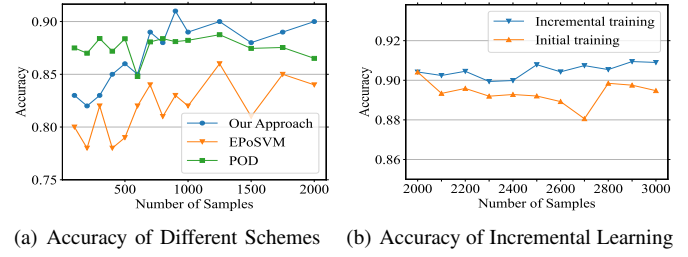


Fig. 2: Comparison of Accuracy

sample size is 2000, demonstrating its effectiveness in training SVM for datasets with large sample sizes and non-linear data distributions. In contrast, the POD and EPoSVM approaches are unable to achieve high accuracy. When the sample size is small, the accuracy of our scheme is comparable to that of EPoSVM approach, but slightly lower than that of the POD approach. This is because when the sample size is small, the samples are approximately linear, and the POD approach, which updates each parameter independently using the SMO method, can achieve higher accuracy. However, the slight increase in accuracy comes with huge computation overhead.

Accuracy of Incremental learning: We measure the accuracy during incremental learning, we simulate a real-world scenario, as initially given 2000 training samples, and after that only 100 incremental data each time. Accuracy of incremental learning is shown in Fig.2(b). Incremental learning results in a slight improvement in the accuracy of trained model. This may be because incremental learning uses a relatively small learning rate, which result in a proper fitting. However, the difference in accuracy is too small, we can only conclude that incremental learning does not decrease the accuracy of the trained model. Thus, the utilization of incremental learning is a reasonable approach.

B. Efficiency Evaluation:

In this part, we evaluate the computation overhead of our scheme along with POD and EPoSVM. In addition, we also simulate a real-world scenario where the data gradually arrives and test the computational cost of our scheme in this scenario. We compare the computational cost of our scheme with that of standard SVM training under this scenario.

Efficiency of training: We measure the computational cost of our scheme, POD, and EPoSVM during the training process for different training samples, as depicted in Fig. 3(a). Notably, we include the computational cost of both the offline and online phases in our comparison of computational efficiency, which is the actual running time to train a SVM using training data. Our scheme significantly improves computational efficiency compared to both EPoSVM and POD. The average training time of our scheme is 7.92 seconds, while the average running time of EPoSVM and POD are 42.50 and 297.71 seconds, respectively. Furthermore, our proposed scheme employs exclusively privacy-preserving computation. Unlike standard gradient descent methods, it eliminates the

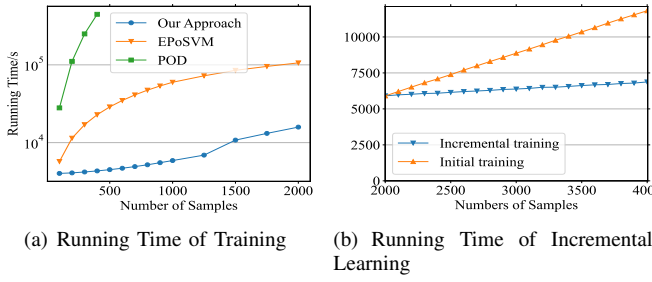


Fig. 3: Computation Overhead of Different Schemes

need for computationally expensive secure division to calculate gradients. This design choice reduces the cost considerably compared to EPoSVM.

Efficiency of incremental learning: We measure the computation overhead of training with different schemes, as illustrated in Fig. 3(b). To ensure a more accurate comparison, we account for the computation overhead of incremental learning along with the overhead of training using 2000 samples. Under real scenario where data arrives steadily, our scheme is capable of reducing the computation overhead as the running time is decreased compared to training SVM with conventional scheme. Incremental learning makes our scheme more practical and useful in real-world scenarios.

Communication Overhead: We measure the total communication overhead of our scheme along with EPoSVM and POD scheme. The experimental results are shown in Table I, notice we only measure the communication overhead in training process, which specifies the communication cost of executing training algorithm for 1 time. We observe that the communication cost of our scheme is slightly decreased compared with EPoSVM. This is mainly due to a variance in system model and basic primitives. In terms of application, the communication of our scheme is acceptable since we assume the cloud servers have sufficient computation and communication resources.

TABLE I: Communication Overhead(KB)

Scheme	n = 10	n = 20	n = 40
Our Scheme	156.74	320.13	641.56
EPoSVM	198.90	403.89	803.21
POD	367.49	992.60	3852.10

VIII. CONCLUSION

In this paper, we proposed an efficient privacy-preserving outsourced training scheme for SVM with non-linear kernel. Our proposed scheme supports privacy-preserving SVM training with non-linear kernel, and enables training with only incremental data. Specifically, we designed several basic computation protocols and developed a training scheme following the idea of gradient descent. Furthermore, we formulated an incremental learning scheme that builds upon our training

methodology. Security analysis shows that our scheme is secure under the semi-honest model, and performance evaluation demonstrates that our scheme is more efficient and achieves higher accuracy than existing schemes. Our scheme is particularly efficient in scenarios where data arrives incrementally.

ACKNOWLEDGMENT

The work is supported in part by Ant Group, the National Natural Science Foundation of China (NSFC) under grants No.61972371 and No. 62372425, and Youth Innovation Promotion Association of the Chinese Academy of Sciences (CAS) under grant No. Y202093.

REFERENCES

- [1] Y. Rahulamathavan, R. C.-W. Phan, S. Veluru, K. Cumanan, and M. Rajarajan, "Privacy-preserving multi-class support vector machine for outsourcing the data classification in cloud," *IEEE Transactions on Dependable and Secure Computing*, vol. 11, no. 5, pp. 467–479, 2014.
- [2] X. Li, Y. Zhu *et al.*, "On the soundness and security of privacy-preserving SVM for outsourcing data classification," *IEEE Transactions on Dependable and Secure Computing*, vol. 15, no. 5, pp. 906–912, 2018.
- [3] E. Osuna, R. M. Freund *et al.*, "Training support vector machines: an application to face detection," *Proceedings of IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pp. 130–136, 1997.
- [4] J. Xu, C. Xu *et al.*, "New incremental learning algorithm with support vector machines," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 49, no. 11, pp. 2230–2241, 2019.
- [5] J. Wang, L. Wu *et al.*, "An efficient and privacy-preserving outsourced support vector machine training for internet of medical things," *IEEE Internet of Things Journal*, vol. 8, no. 1, pp. 458–473, 2021.
- [6] X. Liu, R. H. Deng *et al.*, "Privacy-preserving outsourced support vector machine design for secure drug discovery," *IEEE Transactions on Cloud Computing*, vol. 8, no. 2, pp. 610–622, 2020.
- [7] B. Xie, T. Xiang *et al.*, "Achieving privacy-preserving online diagnosis with outsourced SVM in internet of medical things environment," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 6, pp. 4113–4126, 2022.
- [8] Y. Chen, Q. Mao *et al.*, "Privacy-preserving multi-class support vector machine model on medical diagnosis," *IEEE Journal of Biomedical and Health Informatics*, vol. 26, no. 7, pp. 3342–3353, 2022.
- [9] S. Laur, H. Lipmaa, and T. Mielikäinen, "Cryptographically private support vector machines," in *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2006, pp. 618–624.
- [10] H. Yu, X. Jiang, and J. Vaidya, "Privacy-preserving SVM using nonlinear kernels on horizontally partitioned data," *Proceedings of the 2006 ACM Symposium on Applied Computing*, pp. 603–610, 2006.
- [11] O. L. Mangasarian, E. W. Wild, and G. M. Fung, "Privacy-preserving classification of vertically partitioned data via random kernels," *ACM Trans. Knowl. Discov. Data*, vol. 2, no. 3, pp. 12:1–12:16, 2008.
- [12] D. Rathee, M. Rathee *et al.*, "SiRnn: A math library for secure RNN inference," in *Proceedings of the 2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 1003–1020.
- [13] D. Beaver, "Efficient multiparty protocols using circuit randomization," in *Advances in Cryptology*, 1992, pp. 420–432.
- [14] P. Mohassel and P. Rindal, "ABY3: A mixed protocol framework for machine learning," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 35–52.
- [15] M. S. Riazi, C. Weinert *et al.*, "Chameleon: a hybrid secure computation framework for machine learning applications," in *Proceedings of the 2018 Asia Conference on Computer and Communications Security*, 2018, p. 707–721.
- [16] D. Demmler, T. Schneider *et al.*, "ABY - a framework for efficient mixed-protocol secure two-party computation," in *Network and Distributed System Security Symposium*, 2015.
- [17] A. Patra, T. Schneider *et al.*, "ABY2.0: Improved mixed-protocol secure two-party computation," in *USENIX Security Symposium 2021*, 2021, pp. 2165–2182.